

LINUX + GIT + GITHUB

A safe, visual guide to using three GitHub accounts from one Linux computer

Owner

Rasel Ahammed

Operating system

Linux

Architecture

Personal · Work · Fun

Golden habit

Check the remote. Check the email. Then push.

Edition 1.0 · 31 August 2026

How to use this guide

Start with the mental model, complete setup once, then use the daily checklists.

This edition preserves the substance of the original 74-section guideline, removes repeated explanations, corrects command formatting, and adds preventive automation and recovery guidance.

Security boundary

This document lists **filenames** only. It does not contain any private-key material, passphrases, tokens, passwords, or recovery codes.

Contents

- [01](#) Accounts and the mental model
- [02](#) SSH keys, permissions, and public-key setup
- [03](#) SSH configuration and authentication tests
- [04](#) Git identity: global versus repository-local
- [05](#) Creating, cloning, and connecting projects
- [06](#) Daily Git workflow, branches, merges, and remotes
- [07](#) Secrets, .gitignore, .env, and incident response
- [08](#) Troubleshooting decision trees
- [09](#) Automation that prevents account mistakes
- [10](#) New computer, reboot, passphrases, and maintenance
- [11](#) Master verification and printable cheat sheets
- [12](#) Coverage map for original sections 1–74

01 · Accounts and the mental model

Three account profiles; three SSH keys; one consistent commit name.

Profile	Commit email	SSH alias	Private key	Public key
Personal	rasel82096@gmail.com	github-personal	~/.ssh/id_ed25519_personal	~/.ssh/id_ed25519_personal.pub
Work	ra188789@gmail.com	github-work	~/.ssh/id_ed25519_work	~/.ssh/id_ed25519_work.pub
Fun	rizvi3609@gmail.com	github-fun	~/.ssh/id_ed25519_fun	~/.ssh/id_ed25519_fun.pub

Commit name for all three: Rasel Ahammed

The three independent decisions

Question	Controlled by	Example
Which GitHub account authenticates?	SSH host alias + IdentityFile	git@github-work → Work key
Who appears as commit author?	Git user.name + user.email	Rasel Ahammed <ra188789@gmail.com>
Which repository receives the push?	Remote URL, usually origin	git@github-work:company/project.git

Do not confuse them

Changing user.email does not change the SSH account. Changing the SSH alias does not rewrite commit authorship.

Final mental model

ARCHITECTURE

Repository

- origin → git@github-ALIAS:OWNER/REPO.git
- user.name → Rasel Ahammed
- user.email → profile-specific email

SSH alias → IdentityFile → GitHub account

Git email → author recorded in each new commit

02 · SSH keys and security

Public keys go to GitHub. Private keys never leave the computer.

An Ed25519 key pair contains two related files. GitHub stores the public half; Linux uses the private half to prove possession without sending it.

File	Meaning	May be shared?
id_ed25519_personal	Private key	No — keep secret
id_ed25519_personal.pub	Public key	Yes — add to Personal GitHub
id_ed25519_work	Private key	No — keep secret
id_ed25519_work.pub	Public key	Yes — add to Work GitHub
id_ed25519_fun	Private key	No — keep secret
id_ed25519_fun.pub	Public key	Yes — add to Fun GitHub

The lock-and-key analogy

GitHub stores

The **public key**. It is like a lock that can verify a matching proof.

Linux stores

The **private key**. It creates the proof and must remain secret.

Never do these things

- Send or upload a private key to anyone or to GitHub.
- Paste a private key into a website, chat, ticket, document, or screenshot.
- Commit a private key, .env file, access token, cloud credential, or database password.
- Store private keys in a public or broadly shared cloud folder.

Recommended permissions

TERMINAL

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/config
chmod 600 ~/.ssh/id_ed25519_personal
chmod 600 ~/.ssh/id_ed25519_work
chmod 600 ~/.ssh/id_ed25519_fun
chmod 644 ~/.ssh/id_ed25519_personal.pub
chmod 644 ~/.ssh/id_ed25519_work.pub
chmod 644 ~/.ssh/id_ed25519_fun.pub

ls -la ~/.ssh/
```

Add each public key to the correct GitHub account

Display one public key at a time. Copy the entire single line, sign in to the matching GitHub account, and open **Settings** → **SSH and GPG keys** → **New SSH key**. Choose **Authentication Key**.

DISPLAY PUBLIC KEYS

```
cat ~/.ssh/id_ed25519_personal.pub
cat ~/.ssh/id_ed25519_work.pub
cat ~/.ssh/id_ed25519_fun.pub
```

Public-key file	Destination account
~/.ssh/id_ed25519_personal.pub	Personal
~/.ssh/id_ed25519_work.pub	Work
~/.ssh/id_ed25519_fun.pub	Fun

Important

Each account has its own key pair. Never upload a private-key file. GitHub only needs the line from the .pub file.

Fingerprint the keys

Fingerprints are safe identifiers. Compare them with the fingerprints displayed by GitHub to confirm which key is registered.

FINGERPRINTS

```
ssh-keygen -lf ~/.ssh/id_ed25519_personal.pub
ssh-keygen -lf ~/.ssh/id_ed25519_work.pub
ssh-keygen -lf ~/.ssh/id_ed25519_fun.pub
```

Host keys and known_hosts

On first connection, SSH may ask you to trust GitHub's host key and then stores it in ~/.ssh/known_hosts. Verify GitHub's published host-key fingerprint before accepting when practical. If you later see **REMOTE HOST IDENTIFICATION HAS CHANGED**, investigate; do not delete known_hosts blindly.

03 · SSH configuration

Aliases make the key choice explicit and repeatable.

Save the following in `~/.ssh/config`:

~/.SSH/CONFIG

```
Host github-personal
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519_personal
IdentitiesOnly yes

Host github-work
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519_work
IdentitiesOnly yes

Host github-fun
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519_fun
IdentitiesOnly yes
```

Alias typed in remote	Real host	Forced key
github-personal	github.com	id_ed25519_personal
github-work	github.com	id_ed25519_work
github-fun	github.com	id_ed25519_fun

Why

IdentitiesOnly yes?

It tells SSH to offer the key named for this alias instead of trying unrelated identities from the agent. This reduces accidental account selection and avoids “too many authentication failures.”

Validate the resolved configuration

OPTIONAL DIAGNOSTIC

```
ssh -G github-personal | grep -Ei "^(hostname|user|identityfile|identitiesonly) "
ssh -G github-work | grep -Ei "^(hostname|user|identityfile|identitiesonly) "
ssh -G github-fun | grep -Ei "^(hostname|user|identityfile|identitiesonly) "
```

SSH agent and account tests

The SSH agent caches unlocked private keys. Depending on your desktop keyring, keys may or may not remain available after reboot.

LIST LOADED KEYS

```
ssh-add -l
```

If the agent is unavailable or empty:

START AGENT AND ADD KEYS

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519_personal
ssh-add ~/.ssh/id_ed25519_work
ssh-add ~/.ssh/id_ed25519_fun
ssh-add -l
```

Passphrase input

When Linux asks for a passphrase, nothing appears while you type—not even dots or cursor movement. That is normal. Type it and press Enter.

Test all aliases

AUTHENTICATION TESTS

```
ssh -T git@github-personal
ssh -T git@github-work
ssh -T git@github-fun
```

A successful result normally says: Hi USERNAME! You've successfully authenticated, but GitHub does not provide shell access. The “no shell access” sentence is expected and does not indicate failure.

Verify the username

The username in each response must match the intended Personal, Work, or Fun account. Authentication success to the wrong username is still a configuration problem.

Verbose diagnosis

SHOW OFFERED KEYS

```
ssh -vT git@github-personal
ssh -vT git@github-work
ssh -vT git@github-fun
```

04 · Git commit identity

Use one global name and a repository-local email.

Set the shared author name globally. Intentionally avoid a global email so that a new repository cannot silently inherit the wrong account email.

GLOBAL SETUP

```
git config --global user.name "Rasel Ahammed"
git config --global --unset-all user.email # only if a global email exists
git config --global --list --show-origin
```

Careful with unset

The `--unset-all` command exits with a non-zero status when no global email exists; that is harmless. Review `--show-origin` output to see which file supplies each value.

Set identity inside each repository

Profile	Commands
Personal	<code>git config user.name "Rasel Ahammed" git config user.email "rasel82096@gmail.com"</code>
Work	<code>git config user.name "Rasel Ahammed" git config user.email "ra188789@gmail.com"</code>
Fun	<code>git config user.name "Rasel Ahammed" git config user.email "rizvi3609@gmail.com"</code>

These local settings are stored in `.git/config` and override global values for that repository.

Verify before committing

IDENTITY CHECKS

```
git config user.name
git config user.email
git config --local --list
git config --show-origin --get user.email
```

A commit records the name and email present when the commit is created. Changing Git configuration affects future commits only; it does not automatically repair existing history.

05 · Create and clone projects

The alias inside the remote URL selects the GitHub SSH identity.

Recommended: create an empty repository on the intended GitHub account, then clone using the matching alias.

Personal

```
cd ~/Projects/Personal
git clone git@github-personal:YOUR_PERSONAL_USERNAME/my-project.git
cd my-project
git config user.name "Rasel Ahammed"
git config user.email "rasel82096@gmail.com"
```

Work

```
cd ~/Projects/Work
git clone git@github-work:YOUR_WORK_USERNAME/my-project.git
cd my-project
git config user.name "Rasel Ahammed"
git config user.email "ra188789@gmail.com"
```

Fun

```
cd ~/Projects/Fun
git clone git@github-fun:YOUR_FUN_USERNAME/my-project.git
cd my-project
git config user.name "Rasel Ahammed"
git config user.email "rizvi3609@gmail.com"
```

Replace GitHub's default host

When GitHub shows `git@github.com:OWNER/REPO.git`, replace `github.com` with `github-personal`, `github-work`, or `github-fun`.

After every clone

VERIFICATION

```
git remote -v
git config user.name
git config user.email
git status
```

Start a project locally first

LOCAL-FIRST WORKFLOW

```
mkdir my-project
cd my-project
git init
git branch -M main

# Choose exactly one profile, for example Personal:
git config user.name "Rasel Ahammed"
git config user.email "rasel82096@gmail.com"

# Create files, then review:
git status
git add .
git diff --cached
git commit -m "Initial commit"

# After creating the empty GitHub repository:
git remote add origin git@github-personal:YOUR_USERNAME/my-project.git
git remote -v
git push -u origin main
```

Suggested folder layout

ORGANIZATION

```
~/Projects/
Personal/
invoice-app/
portfolio/
Work/
company-dashboard/
client-api/
Fun/
game-project/
```

Folder names do not enforce identity, but they provide a strong visual clue. The repository-local Git settings and remote URL remain the authoritative controls.

Empty repository tip

When pushing an existing local project for the first time, create the GitHub repository without a README, license, or .gitignore to avoid an unnecessary first-commit divergence. If the remote already has commits, pull and reconcile deliberately.

06 · Daily Git workflow

Review first; commit intentionally; push only after identity checks.

DAILY WORKFLOW

```
cd PROJECT_FOLDER

# Context and identity
git status
git remote -v
git config user.name
git config user.email

# Synchronize
git pull

# Work, then inspect
git diff
git add .
git diff --cached

# Commit and push
git commit -m "Describe what changed"
git push
```

Before an important push

Confirm: (1) this is the correct repository, (2) the remote uses the intended alias, (3) the email matches the profile, and (4) the staged diff contains no secrets or unintended files.

Useful inspection commands

Purpose	Command
Working-tree status	git status
Unstaged changes	git diff
Staged changes	git diff --cached
Remote destinations	git remote -v
Local branches	git branch
All local and remote branches	git branch -a
Compact history	git log --oneline
Graph history	git log --oneline --decorate --graph --all
All effective configuration	git config --list --show-origin
Repository-only configuration	git config --local --list

Staging, restoring, commits, branches, and merges

Stage precisely

```
git add filename
git add file1 file2
git add .
git status
```

Undo staging without deleting work

```
git restore --staged filename
```

Discard an uncommitted file change

```
git restore filename
```

Destructive action `git restore filename` can permanently discard uncommitted work. Inspect `git diff -- filename` first.

Commit messages

Prefer an imperative, specific summary: Add invoice upload form, Fix authentication error, or Add dashboard filters. Avoid vague messages such as “update,” “stuff,” or “changes.”

Branches

FEATURE BRANCH

```
git switch -c invoice-upload
# work, stage, commit
git push -u origin invoice-upload

git switch main
git branch
```

Merge locally when appropriate

```
git switch main
git pull
git merge invoice-upload
git push
```

For shared projects, a GitHub pull request is often safer because it supports review, checks, and discussion before merging.

Remotes, first push, and pulling safely

A remote is a named repository destination. `origin` is the conventional name created by `clone`.

INSPECT

```
git remote -v
```

Correct a wrong remote

```
# Personal
git remote set-url origin git@github-personal:USERNAME/REPOSITORY.git

# Work
git remote set-url origin git@github-work:USERNAME/REPOSITORY.git

# Fun
git remote set-url origin git@github-fun:USERNAME/REPOSITORY.git

git remote -v
```

First push of a branch

```
git push -u origin main
```

The `-u` option records the upstream branch. Later, `git push` and `git pull` know which remote branch to use.

Pulling and conflicts

Use `git pull` before starting work on a shared branch. If Git reports conflicts, do not blindly delete files. Open each conflicted file, understand both sides, resolve markers carefully, test, stage the resolutions, and finish the merge or rebase.

Safer shared workflow

Prefer feature branches and pull requests. Avoid force-pushing shared branches unless the team has explicitly agreed and you understand the consequences.

07 · Secrets and repository hygiene

Prevent exposure before it reaches Git history.

A `.gitignore` file tells Git which untracked files and folders to ignore.

EXAMPLE .GITIGNORE

```
.env
.env.*
!.env.example

node_modules/
__pycache__/*
*.pyc

dist/
build/
coverage/

.vscode/
.DS_Store
```

Use a safe environment template

.ENV .EXAMPLE

```
DATABASE_PASSWORD=
API_KEY=
JWT_SECRET=
```

Never commit

- Passwords, API keys, access tokens, session secrets, or database credentials.
- SSH private keys or cloud-provider credential files.
- Real `.env` files or production configuration containing secrets.

Check what Git will record

BEFORE COMMIT

```
git status
git diff
git diff --cached
```

Already tracked?

`.gitignore` does not untrack a file already committed. After protecting and rotating any secret, remove the file from the index with `git rm --cached FILE`, then commit the removal. Historical copies still require history cleanup.

If a secret or SSH key is exposed

First response

Rotate or revoke the credential immediately. Deleting the file in a later commit is not enough because earlier Git history may still contain it.

General secret response

- Revoke or rotate the password, token, API key, or database credential.
- Remove it from the current repository state and add an appropriate ignore rule.
- Assess whether it exists in tags, branches, forks, CI logs, releases, caches, or package artifacts.
- Clean Git history when necessary using an approved tool and coordinate any force-push with collaborators.
- Review GitHub secret-scanning and security alerts.
- Generate and distribute only a new credential; never reuse the exposed value.

Private SSH key response

- Sign in to the matching GitHub account and remove the corresponding public key.
- Generate a new key pair on the trusted computer.
- Add only the new public key to GitHub.
- Update `~/.ssh/config` if the filename changes.
- Test the alias and verify the returned username.
- Delete the old private key only after the replacement is confirmed.

Account hardening

- Enable two-factor authentication on all three accounts.
- Store recovery codes securely and separately.
- Use strong, unique account passwords.
- Review authorized SSH keys periodically.
- Remove keys for old computers or any device you no longer control.

08 · Troubleshooting decision trees

Diagnose the layer that failed instead of regenerating keys randomly.

Permission denied (publickey)

FIRST CHECKS

```
ssh-add -l
ssh -T git@github-personal
ssh -vT git@github-personal
```

- Confirm the alias is spelled correctly in the remote and command.
- Confirm the alias maps to the intended IdentityFile.
- Confirm the private key exists and has restrictive permissions.
- Confirm the matching public key is registered on the intended GitHub account.
- Read verbose output to see which key is offered and whether GitHub accepts it.

Repository not found

```
git remote -v
ssh -T git@github-work
```

- Check owner/organization and repository spelling.
- Confirm the repository exists and your authenticated account has access.
- Confirm a private repository is being accessed with the intended account.
- Confirm the remote uses the correct Personal, Work, or Fun alias.

Authenticated as the wrong account

```
ssh -T git@github-personal
ssh -T git@github-work
ssh -T git@github-fun
git remote -v
```

Compare each returned GitHub username with the intended account, then repair the alias configuration or remote URL.

Wrong email, wrong remote, host warnings, and conflicts

Wrong email for future commits

```
# Personal
git config user.email "rasel82096@gmail.com"
# Work
git config user.email "ra188789@gmail.com"
# Fun
git config user.email "rizvi3609@gmail.com"

git config user.email
```

Wrong remote

```
git remote -v
git remote set-url origin CORRECT_URL
git remote -v
```

Remote host identification changed

Stop and investigate. Verify GitHub's current host-key announcement from an independent trusted source. Do not bypass the warning or erase `known_hosts` merely to make the message disappear.

Merge conflict

- Run `git status` to list conflicted files.
- Open each file and resolve <<<<<<, =====, and >>>>>> sections.
- Test the result, then `git add` each resolved file.
- Finish with the merge/rebase command Git requests, or abort deliberately if needed.

Do not guess

If a command fails, capture the exact command and complete error message. Diagnose SSH authentication, repository authorization, Git identity, and network/host verification as separate layers.

09 · Prevent mistakes automatically

Optional safeguards reduce repeated manual configuration.

Option A — conditional Git configuration by folder

This automatically selects an account-specific config based on the repository's directory. The trailing slash and `**` matter.

`~/ .GITCONFIG`

```
[user]
name = Rasel Ahammed

[includeIf "gitdir:~/Projects/Personal/**"]
path = ~/.gitconfig-personal
[includeIf "gitdir:~/Projects/Work/**"]
path = ~/.gitconfig-work
[includeIf "gitdir:~/Projects/Fun/**"]
path = ~/.gitconfig-fun
```

`~/ .GITCONFIG-PERSONAL`

```
[user]
email = rasel82096@gmail.com
```

`~/ .GITCONFIG-WORK`

```
[user]
email = ra188789@gmail.com
```

`~/ .GITCONFIG-FUN`

```
[user]
email = rizvi3609@gmail.com
```

Verify the match

Inside one repository from each folder, run `git config --show-origin --get user.email`. If a path does not match, Git may have no email—which is safer than silently using the wrong one.

More optional guardrails

Fail commits when identity is missing

```
git config --global user.useConfigOnly true
```

With `user.useConfigOnly=true`, Git avoids guessing an identity from the system username and hostname. A repository must receive a valid email from local or conditional configuration.

Pre-push sanity script

CHECK-GIT-CONTEXT

```
#!/usr/bin/env bash
set -euo pipefail
printf "Remote: %s\n" "$(git remote get-url origin)"
printf "Name: %s\n" "$(git config user.name)"
printf "Email: %s\n" "$(git config user.email)"
git status --short
```

Save it somewhere on your PATH, make it executable, and run it before important pushes. A stricter hook can block mismatched alias/email combinations, but test hooks carefully and keep the rule understandable.

Repository-specific SSH command — advanced fallback

```
git config core.sshCommand "ssh -i ~/.ssh/id_ed25519_work -o IdentitiesOnly=yes"
```

This pins a key inside one repository. It can be useful for exceptional cases, but aliases are easier to inspect in `git remote -v`. Avoid layering both approaches unless you document why.

Recommended baseline

Use SSH aliases + folder organization + conditional Git email + `user.useConfigOnly=true`. Keep the system simple enough that you can explain every control.

10 · Reboot, passphrases, and a new computer

After reboot

```
ssh-add -l

# If needed:
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519_personal
ssh-add ~/.ssh/id_ed25519_work
ssh-add ~/.ssh/id_ed25519_fun
ssh-add -l
```

A desktop keyring can remember passphrases and load keys automatically. Configure it only after the basic workflow is stable and verify that aliases still select the intended identity.

Forgotten passphrase

There is no normal command that reveals a forgotten private-key passphrase. If you cannot recover it, create a replacement key pair and add the new public key to GitHub. Do not delete a working key until its replacement is tested.

Adding a new Linux computer

Generating new keys on the new computer is generally safer than casually copying private keys between machines.

EXAMPLE

```
ssh-keygen -t ed25519 \
-C "rasel82096@gmail.com" \
-f ~/.ssh/id_ed25519_personal

# Repeat with the Work and Fun email/filename.
```

- Add each new public key to the matching GitHub account.
- Re-create `~/.ssh/config` and permissions.
- Test all three aliases and verify returned usernames.
- Configure Git identity and optional conditional includes.
- Remove obsolete keys from computers you no longer control.

Optional future features

Commit signing

Git can sign commits with SSH keys or other mechanisms, producing “Verified” commits when configured correctly. This is optional and separate from SSH transport authentication. Add it only after the normal clone/commit/push workflow is stable; consider separate signing keys or carefully document reused keys.

GitHub CLI

The GitHub CLI (gh) can manage pull requests, issues, workflows, and repositories. Its login/session selection is another identity layer, separate from Git’s SSH remote. If you use it with multiple accounts, always verify the active gh account before actions.

Credential and access review

- Review SSH keys in every GitHub account on a schedule.
- Remove stale devices and expired access.
- Verify 2FA and recovery methods.
- Audit repository remotes after reorganizing folders or transferring ownership.
- Keep operating-system security updates current.

Keep complexity proportional

More automation can help, but every additional identity mechanism can create a new failure mode. Prefer controls that are visible in ordinary checks: remote URL, local email, SSH alias, and test result.

11 · Master verification

Run this sequence whenever you want to confirm the complete setup.

MASTER VERIFICATION

```
# 1. Files and permissions
ls -la ~/.ssh/

# 2. Alias definitions
cat ~/.ssh/config

# 3. Loaded keys
ssh-add -l

# 4. Account authentication
ssh -T git@github-personal
ssh -T git@github-work
ssh -T git@github-fun

# 5. Global Git settings
git config --global --list --show-origin

# 6. Inside a repository
git remote -v
git config user.name
git config user.email
git config --show-origin --get user.email
git status
```

Expected results

- Each alias returns the intended GitHub username.
- Global name is Rasel Ahammed. No accidental global email overrides the profile strategy.
- The repository remote contains the intended alias.
- The effective repository email matches Personal, Work, or Fun.
- Private-key and SSH config permissions are restricted.
- The staged diff contains no secrets or unintended files.

Healthy setup

Authentication account + commit email + repository destination all match the project's purpose.

Printable daily cheat sheet

Moment	Commands / question
Start	cd PROJECT git status
Check identity	git remote -v git config user.name git config user.email
Get latest	git pull
Review work	git diff
Stage	git add .
Review staged	git diff --cached
Commit	git commit -m "Describe the change"
Push	git push
Ask yourself	Correct repository? Correct alias? Correct email? No secrets?

Quick clone

```
git clone git@github-personal:USERNAME/REPOSITORY.git
git clone git@github-work:USERNAME/REPOSITORY.git
git clone git@github-fun:USERNAME/REPOSITORY.git
```

Quick SSH test

```
ssh -T git@github-personal
ssh -T git@github-work
ssh -T git@github-fun
```

Printable troubleshooting cheat sheet

Problem	Run first
SSH problem	<code>ssh-add -l ssh -T git@github-ALIAS ssh -vT git@github-ALIAS</code>
Wrong account	<code>git remote -v ssh -T git@github-personal ssh -T git@github-work ssh -T git@github-fun</code>
Wrong email	<code>git config user.email git config --show-origin --get user.email</code>
Wrong remote	<code>git remote -v git remote set-url origin CORRECT_URL</code>
Agent empty	<code>eval "\$(ssh-agent -s)" ssh-add ~/.ssh/id_ed25519_personal ssh-add ~/.ssh/id_ed25519_work ssh-add ~/.ssh/id_ed25519_fun</code>
Repository not found	<code>git remote -v ssh -T git@github-ALIAS</code>
Host key changed	Stop; verify GitHub host-key information independently

Remember

Do not regenerate keys randomly. Identify whether the failure is key availability, alias mapping, GitHub key registration, repository access, remote spelling, or commit identity.

Golden rules

- Personal = `git@github-personal:... + rase182096@gmail.com`.
- Work = `git@github-work:... + ra188789@gmail.com`.
- Fun = `git@github-fun:... + rizvi3609@gmail.com`.
- Never share private SSH keys.
- Never commit secrets. Rotate immediately after exposure.
- Check the remote. Check the email. Then push.

12 · Coverage map

Where the original 74 sections appear in this redesigned edition.

Original sections	Included topics	Location
Original 1–5	Accounts, SSH vs Git identity, key files, security, lock-and-key model	Chapters 01–02
Original 6–11	SSH config, permissions, agent, tests, debugging, GitHub key setup	Chapters 02–03
Original 12–14	Global and local Git identity; verification	Chapter 04
Original 15–22	Create and clone Personal, Work, and Fun projects; alias rule	Chapter 05
Original 23–36	Daily workflow, pre-push checks, remotes, staging, commits, pull, branches, merges	Chapter 06
Original 37–40	gitignore, env files, staged review, secret response	Chapter 07
Original 41–50	known_hosts, permissions, fingerprints, SSH and repository troubleshooting, URL rewriting	Chapters 02, 03, 08
Original 51–56	Complete project examples, folders, before-work and before-push habits	Chapters 05–06, cheat sheets
Original 57–64	Reboot, passphrases, new computer, compromised keys, account security, signing, GitHub CLI	Chapters 07 and 10
Original 65–68	Global/local checks, master verification, golden rules	Chapters 04 and 11
Original 69–74	Daily, clone, SSH, troubleshooting cheat sheets, mental model, essential commands	Chapters 01 and 11

Editorial note

Repeated commands were consolidated rather than omitted. Additional material includes config-origin checks, conditional includes, `user.useConfigOnly`, safer conflict guidance, key fingerprinting, and a pre-push context script.

End state

```
Personal → github-personal → id_ed25519_personal → rasel82096@gmail.com
Work → github-work → id_ed25519_work → ra188789@gmail.com
Fun → github-fun → id_ed25519_fun → rizvi3609@gmail.com
```

```
Commit name for all: Rasel Ahammed
```

CHECK THE REMOTE. CHECK THE EMAIL. THEN PUSH.